

中国科学技术大学

实验报告

计算机图形

学: Computer Graphics

实验六: ARAP Mesh Parameterization

专业班级 2023 级计算机图形学

学 号 PB23000052

姓 名 杨硕

指导教师 刘利刚

日 期 2026 年 04 月 19 日

中国科学技术大学

目 录

1 实验概述	1
2 实验任务与节点接口	1
3 算法原理	2
3.1 初始参数化	2
3.2 三角形局部坐标与 Jacobian	3
3.3 ARAP 能量	3
3.4 Local Phase	4
3.5 Global Phase	4
4 可选模型	5
4.1 ASAP 模型	5
4.2 Hybrid 模型	5
5 实现细节与鲁棒性	5
6 实验结果	6
6.1 ARAP 迭代过程	6
6.2 ASAP、Hybrid 与 ARAP 对比	6
6.3 Hybrid 参数消融	7
6.4 棋盘纹理验证	7
6.5 翻转后处理	7
6.6 定量指标	8
6.7 Local Phase 并行化	8
7 结果分析	9
8 思考题与拓展讨论	9
8.1 曲面的微分坐标理解	9
8.2 参数化后的进一步应用	10

8.3 非同胚于盘的情况	10
9 AI 辅助说明	10
10 总结	11
11 附录: AI 辅助过程记录	12
11.1 作业理解与实现范围	12
11.2 构建依赖问题	12
11.3 程序启动与节点使用	12
11.4 卡死问题定位与优化	13
11.5 报告截图与定量指标	13
11.6 报告写作	14

1 实验概述

本次实验实现了基于 Sorkine 等人提出的 local/global 框架的 ARAP (As-Rigid-As-Possible) 网格参数化方法。给定三角网格的三维几何位置，算法首先构造一个初始二维参数化，然后在每个三角形上交替执行局部刚性拟合和全局稀疏线性系统求解，使二维展开尽量保持原三维三角形的局部形状。

除基础 ARAP 外，本实验还实现并测试了 ASAP (As-Similar-As-Possible)、Hybrid 模型、翻转三角形后处理、畸变指标输出、local phase 并行化，以及多组参数的消融对比。为了避免上一次作业中被指出的程序效率问题，本次所有全局线性系统均使用 Eigen 稀疏矩阵和预分解求解，网格数据主要通过 Ruzino 的组件拷贝和局部变量管理，避免裸指针生命周期不清的问题。

2 实验任务与节点接口

本实验主要实现文件为 Framework3D/Ruzino/source/Editor/geometry_nodes/hw6_arap.cpp。实现的节点如下：

- hw6_arap: 固定使用 ARAP 模型，输出三维带 UV 网格、二维展开网格、UV 数组和误差指标。
- hw6_asap: 固定使用 ASAP 模型，使用一次全局最小二乘求解相似变换意义下的参数化。
- hw6_hybrid: 实现论文中的 Hybrid 模型，使用参数 Lambda 在 ASAP 与 ARAP 行为之间调节。
- hw6_parameterization: 统一入口节点，通过 Mode 在 ASAP、Hybrid、ARAP 三种模式之间切换。

图 图 2-1 展示了本实验用于测试的典型节点图。read_usd 读入网格后同时连接到 Input 与 Original Mesh; hw6_arap.Flat 输出二维展开结果，接入 write_usd.Geometry 写回当前 USD Stage。实验中使用不同 Sub Path 保存不同参数下的结果，避免覆盖原始网格。

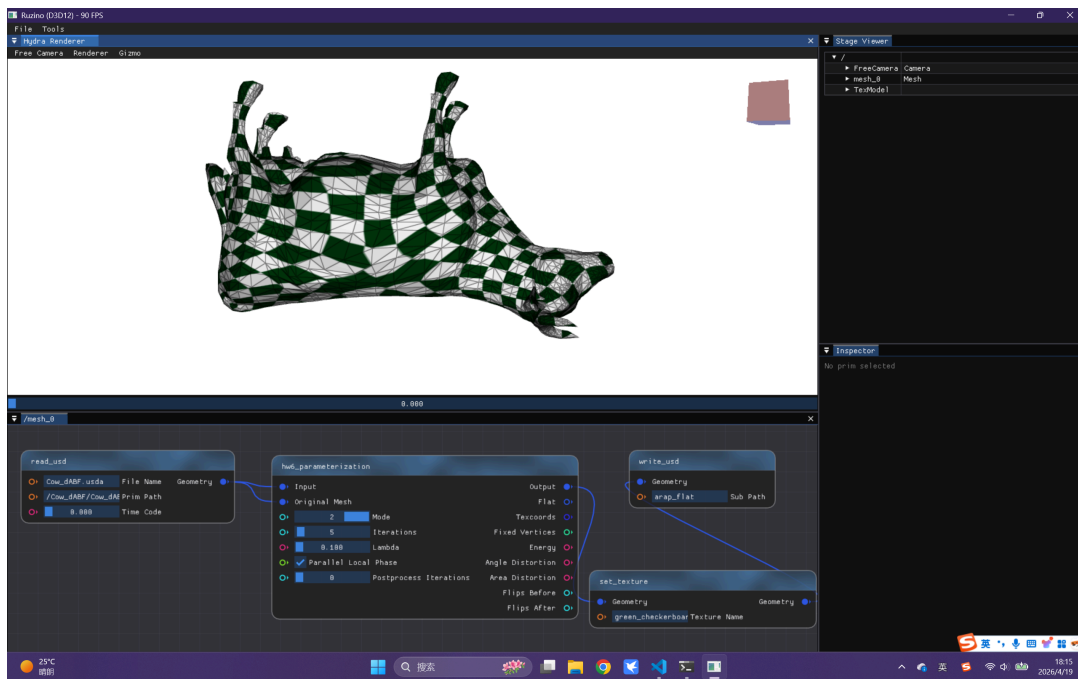


图 2-1 ARAP 参数化节点图。输入网格同时作为当前参数化网格与原始几何网格，Flat 输出用于观察二维展开结果。

3 算法原理

3.1 初始参数化

ARAP local/global 迭代需要一个初始二维参数化 $u_i = (u_i, v_i)$ 。本实验支持两类初始输入：

- 若输入网格已有逐顶点 UV，则直接使用该 UV 作为初始值；
- 若输入网格没有可用 UV，则使用三维包围盒最长的两个坐标轴作平面投影，并归一化到单位方形。

设顶点位置为 $p_i \in \mathbb{R}^3$ ，选择包围盒尺度最大的两个轴 a, b ，则初始 UV 为

$$u_i = \left(\frac{p_i^a - \min_j p_j^a}{\max_j p_j^a - \min_j p_j^a}, \frac{p_i^b - \min_j p_j^b}{\max_j p_j^b - \min_j p_j^b} \right). \quad (3.1)$$

这种初始化速度为 $O(n)$ ，适合在交互式节点系统中使用。它不追求最终质量，只提供一个稳定起点，后续形状保持由 ARAP 迭代完成。

图 图 3-1 展示了 Cow 模型的一次初始展开结果。

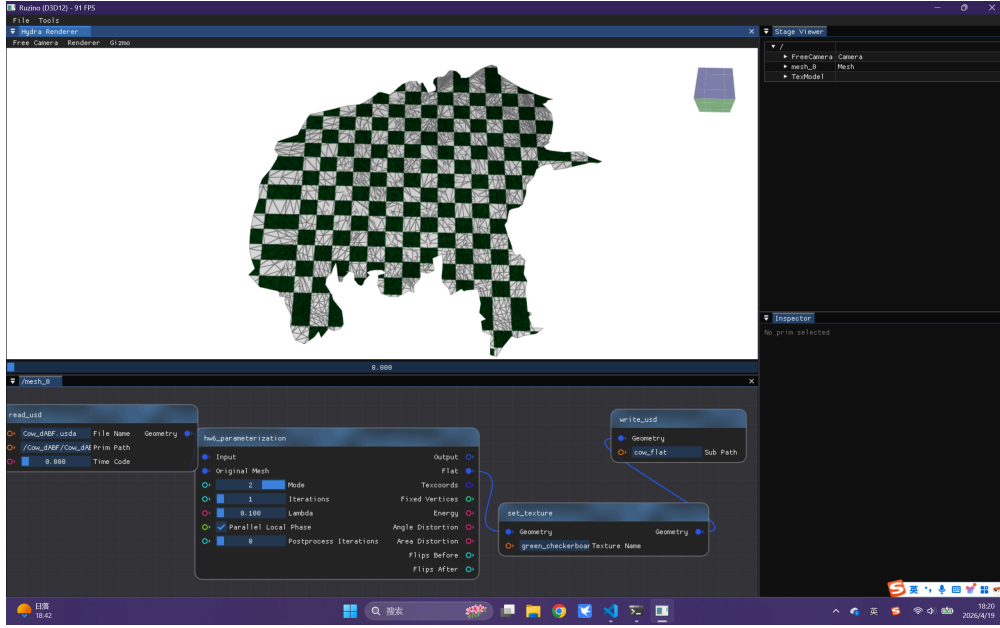


图 3-1 Cow 模型的初始参数化结果。实验中没有读取到可用 texcoord 时，使用主平面投影构造初始 UV。

3.2 三角形局部坐标与 Jacobian

对每个三角形 $t = (i, j, k)$ ，先在三维空间中构造局部二维坐标 $x_i, x_j, x_k \in \mathbb{R}^2$ 。令 $x_i = (0, 0)$ ， $x_j = (|p_j - p_i|, 0)$ ， x_k 由边长投影得到。这样每个三角形的三维形状被等距表示在局部二维平面中。

在三角形内部，参数化映射是分片仿射的。其 Jacobian 为

$$J_t = \sum_{r \in t} u_r \nabla \varphi_r^T, \quad (3.2)$$

其中 φ_r 是局部二维三角形上的线性基函数。若记 $D_t = [x_j - x_i, x_k - x_i]$ ， $U_t = [u_j - u_i, u_k - u_i]$ ，也可以写成

$$J_t = U_t D_t^{-1}. \quad (3.3)$$

3.3 ARAP 能量

ARAP 希望每个三角形的参数化 Jacobian 尽可能接近一个旋转矩阵。设 A_t 为三角形面积， $L_t \in \text{SO}(2)$ 为局部最优旋转，则能量为

$$E(u, L) = \sum_t A_t \|J_{t(u)} - L_t\|_F^2. \quad (3.4)$$

该能量关于 u 与 L 联合非线性，但固定其中一组变量时可以高效求解，因此采用 local/global 交替优化。

3.4 Local Phase

固定当前 UV 坐标 u ，每个三角形独立求解

$$L_t = \arg \min_{R \in \text{SO}(2)} \|J_t - R\|_F^2. \quad (3.5)$$

理论上可通过 2×2 SVD 或极分解得到。若

$$J_t = U \Sigma V^T, \quad (3.6)$$

则

$$L_t = UV^T. \quad (3.7)$$

本实现使用等价的二维解析极分解。对矩阵

$$J = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad (3.8)$$

令

$$x = a + d, \quad y = c - b, \quad (3.9)$$

则最近旋转为

$$R = \frac{1}{\sqrt{x^2 + y^2}} \begin{pmatrix} x & -y \\ y & x \end{pmatrix}. \quad (3.10)$$

这样避免了在每个三角形、每次迭代中调用通用 SVD，减少了交互式运行时的计算开销。

3.5 Global Phase

固定所有 L_t 后，优化变量只剩顶点 UV。目标函数为二次型：

$$\min_u \sum_t A_t \left\| \sum_{r \in t} u_r \nabla \varphi_r^T - L_t \right\|_F^2. \quad (3.11)$$

对每个自由顶点求导，得到稀疏线性系统

$$Ku_x = b_x, \quad Ku_y = b_y, \quad (3.12)$$

其中

$$K_{i,j} = \sum_{t:i,j \in t} A_t \nabla \varphi_i^T \nabla \varphi_j. \quad (3.13)$$

矩阵 K 只由原始网格几何与固定顶点决定，在迭代过程中保持不变。因此本实现只构造并预分解一次稀疏矩阵，之后每轮迭代只更新右端项并求解两个线性

系统。实现中使用 `Eigen::SparseMatrix` 与 `Eigen::SimplicialLLT`，避免稠密矩阵带来的内存和时间开销。

为消除平移和旋转自由度，实验中选择两个距离较远的边界顶点作为固定点。若网格边界不足，则退化为在全部顶点中选择两点。

4 可选模型

4.1 ASAP 模型

ASAP 模型将局部变换限制为相似变换，而不是纯旋转。二维相似变换可表示为

$$L = \begin{pmatrix} a & b \\ -b & a \end{pmatrix}. \quad (4.1)$$

其能量为

$$E_{\text{ASAP}} = \sum_t A_t \left\| J_t - \begin{pmatrix} a_t & b_t \\ -b_t & a_t \end{pmatrix} \right\|_F^2. \quad (4.2)$$

本实现将每个三角形的相似变换参数与顶点 UV 一起放入一次全局最小二乘系统中求解，用作与 ARAP 的对比。

4.2 Hybrid 模型

Hybrid 模型在 ASAP 的相似变换基础上加入对面积缩放的约束。直观上，ASAP 更强调角度保持，ARAP 更强调局部刚性，Hybrid 通过参数 λ 控制二者之间的折中。实验中测试了 $\lambda = 0, 1, 100$ 三种情况，观察从 ASAP-like 到 ARAP-like 的变化趋势。

5 实现细节与鲁棒性

本实验针对上次作业反馈中的程序短板作了专门处理：

- 稀疏矩阵：ARAP global phase 和 harmonic 备用初始化均使用 `Eigen::SparseMatrix`，主流程使用 `SimplicialLLT` 预分解；ASAP 使用稀疏最小二乘正规方程。
- 生命周期管理：输出几何通过 Ruzino 组件的 copy 机制构造，网格临时数据保存在局部 `std::vector` 与结构体中，避免手动管理裸指针。

- 交互性能：local phase 可开启多线程；ARAP 旋转拟合与畸变评估使用二维解析公式，避免大量小矩阵 SVD。
- 缓存：节点会根据输入几何、模式、迭代次数、 λ 、后处理次数等生成签名，重复执行同一配置时直接复用结果。
- 翻转处理：输出 Flips Before 与 Flips After，并提供后处理迭代参数，便于观察翻转三角形数量变化。

6 实验结果

6.1 ARAP 迭代过程

图 6-1 展示了 Cow 模型在不同迭代次数下的 ARAP 展开结果。可以看到，从 1 次迭代到 20 次迭代，整体形状逐渐稳定；局部三角形的刚性约束不断被全局系统吸收，展开结果趋向于更平滑的低畸变形态。

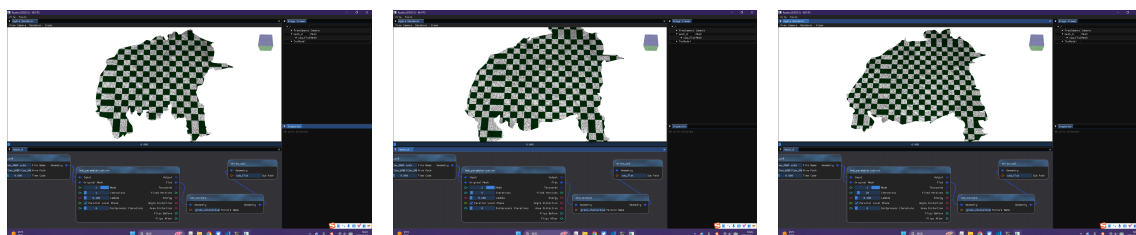


图 6-1 ARAP 迭代次数消融：从左到右分别为 1、5、20 次迭代。随着迭代增加，二维展开逐步收敛。

6.2 ASAP、Hybrid 与 ARAP 对比

图 6-2 比较了 ASAP、Hybrid 和 ARAP 三种模型。ASAP 允许局部相似缩放，因此更偏向角度保持；ARAP 只允许旋转，局部刚性更强；Hybrid 介于二者之间，通过 λ 调节面积缩放惩罚。

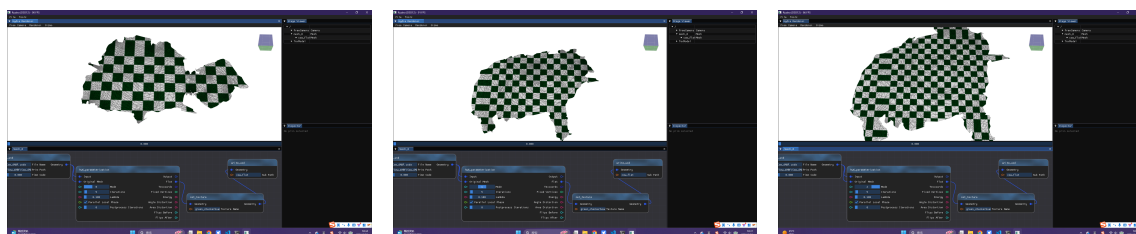


图 6-2 不同参数化模型对比：左为 ASAP，中为 Hybrid，右为 ARAP。三者的局部变换约束不同，因此展开形状存在可见差异。

6.3 Hybrid 参数消融

图 6-3 展示了 Hybrid 模型在不同 λ 下的展开结果。 λ 越大，对局部面积缩放的惩罚越强，结果越接近 ARAP； λ 较小时，结果更接近 ASAP 的相似变换优化。

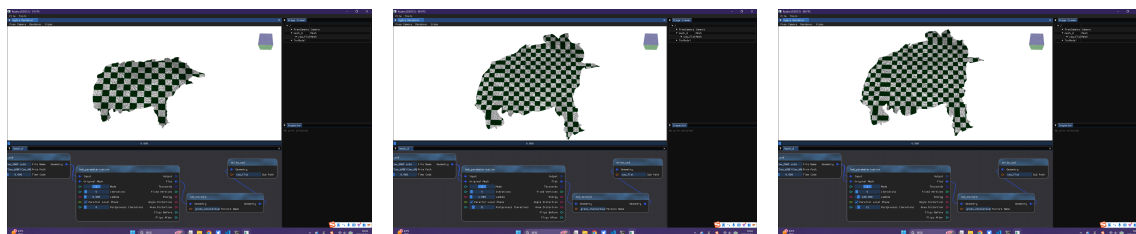


图 6-3 Hybrid 模型参数消融：从左到右分别为 $\lambda = 0$ 、 $\lambda = 1$ 、 $\lambda = 100$ 。

6.4 棋盘纹理验证

图 6-4 展示了将参数化结果用于三维模型纹理映射后的效果。棋盘纹理可以直观反映局部畸变：若参数化中某区域角度或面积变化过大，棋盘格会表现为明显拉伸、压缩或扭曲。

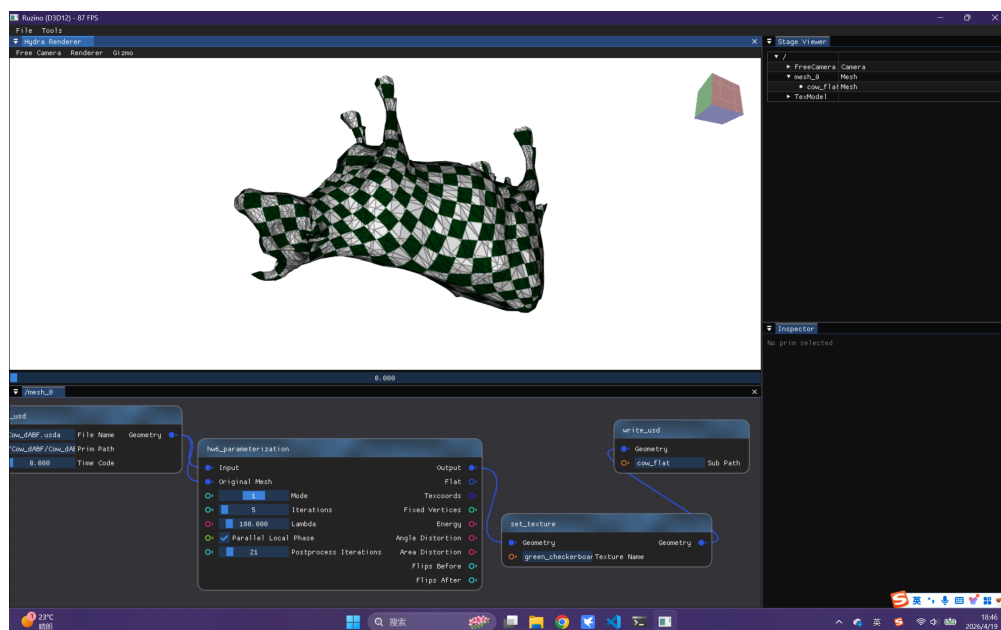


图 6-4 ARAP 参数化用于 Cow 模型棋盘纹理映射。该结果用于观察局部角度和面积畸变。

6.5 翻转后处理

图 6-5 展示了关闭与开启翻转后处理时的结果。报告中同时记录节点输出的 Flips Before 与 Flips After，用于说明后处理是否减少了翻转三角形。翻转处理的目的是不是重新求全局最优，而是在已有参数化附近做局部修正，提高结果的可用性。

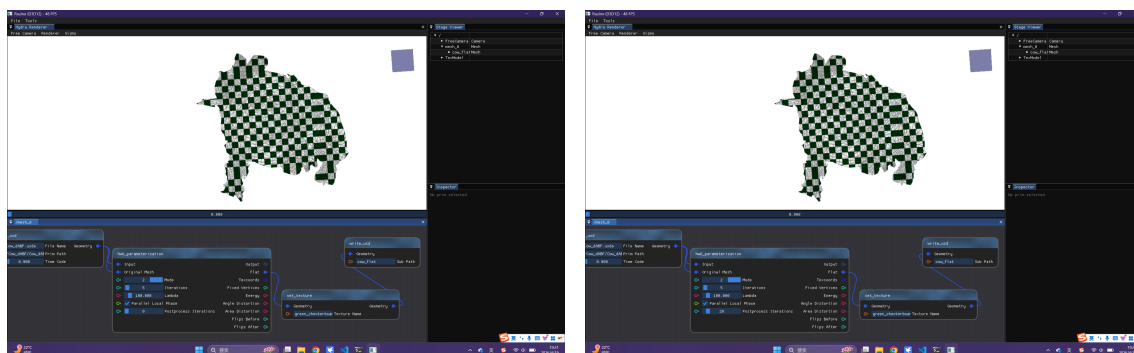


图 6-5 翻转后处理对比：左为 Postprocess Iterations = 0，右为 Postprocess Iterations = 20。

6.6 定量指标

实验中节点输出并在控制台打印以下指标：

- Energy: 当前模型能量，衡量 J_t 与局部最优变换 L_t 的距离；
- Angle Distortion: 由 Jacobian 奇异值比值衡量的角度畸变；
- Area Distortion: 由 Jacobian 面积缩放衡量的面积畸变；
- Flips Before / After: 后处理前后的翻转三角形数量。

图 图 6-6 为一次实验的控制台指标输出。该日志由代码在求解结束后自动打印，便于复现实验表格和报告截图。

```
[19:45:48] : HW6 Parameterization: no usable per-vertex texcoords; using
planar projection initialization.
[19:45:48] : HW6 Parameterization: vertices=3195, triangles=5804, mode=2
, iterations=5
[19:47:43] : HW6 Parameterization Metrics: mode=2, iterations=5, lambda=
100000000, energy=0.46845076, angle_distortion=3008.3499, area_distortio
n=3009.0953, flips_before=35, flips_after=12, fixed_vertices=(1860, 2926
)
```

图 6-6 ARAP 节点输出的能量、角度畸变、面积畸变和翻转三角形数量。

6.7 Local Phase 并行化

Local phase 中每个三角形的最优局部变换互不依赖，因此天然适合并行。图 图 6-7 展示了关闭和开启并行开关时的实验设置。对于更大模型，并行 local phase 能减少每轮迭代中局部拟合的耗时；对于较小模型，线程调度开销可能抵消部分收益。

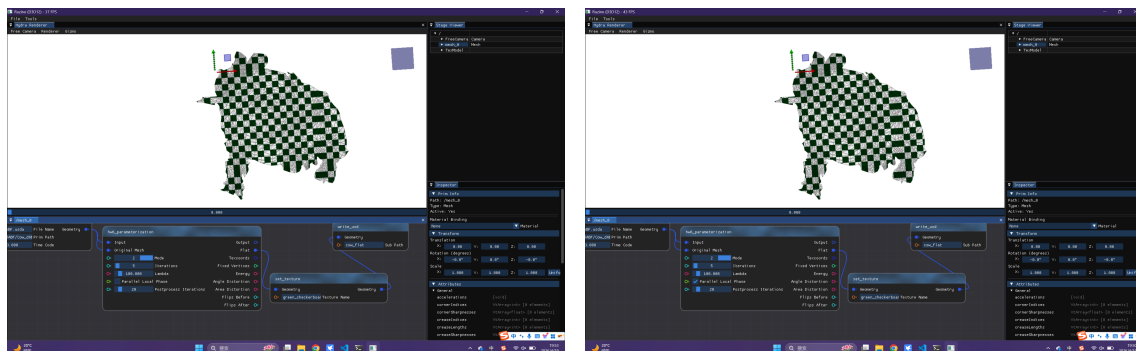


图 6-7 Local phase 并行开关对比：左为关闭，右为开启。

7 结果分析

从实验结果可以观察到以下现象。

首先，ARAP 的迭代过程具有明显的 local/global 收敛特征。初始投影只提供粗糙起点，经过多轮局部旋转拟合和全局稀疏求解后，展开结果更加稳定。由于每轮 global phase 精确最小化固定 L_t 下的二次能量，local phase 又为每个三角形选择当前最近旋转，整体能量通常呈下降趋势。

其次，ASAP、Hybrid、ARAP 的差异符合理论预期。ASAP 允许局部缩放，因而更接近保角参数化；ARAP 强制局部只做旋转，保刚性更强但可能牺牲部分面积分布；Hybrid 通过 λ 在二者之间调节，适合在角度和面积之间做折中。

再次，稀疏矩阵和预分解对交互式程序非常重要。如果每轮都重新组装和分解全局矩阵，或者使用稠密线性代数，节点会在较大模型上明显卡顿。本实现将固定系数矩阵只分解一次，同时加入输入缓存，避免节点图刷新时重复计算相同配置。

最后，翻转三角形是参数化任务中必须检查的问题。单纯降低 ARAP 能量不一定保证完全无翻转，因此报告中同时展示几何结果和 Flips Before / After 指标，以避免只凭视觉结果判断算法质量。

8 思考题与拓展讨论

8.1 曲面的微分坐标理解

曲面参数化中的 Jacobian J_t 可以看作从原三维曲面的局部切平面到二维参数域的微分映射。它描述了一个无穷小向量在参数化前后的拉伸、旋转和剪切。ARAP 并不直接约束顶点位置的绝对误差，而是约束每个局部微分映射尽量接近旋转，因此它更关注局部形状保持。

若 J_t 的两个奇异值接近 1，则局部长度和面积保持较好；若两个奇异值比例接近 1，但绝对值不为 1，则更接近保角但存在面积缩放；若奇异值相差很大，则局部角度畸变明显。

8.2 参数化后的进一步应用

除纹理贴图外，参数化还可用于在曲面上定义二维信号处理任务。例如可以把三维网格展开到二维域后，在参数域中进行规则网格采样、局部编辑、纹理合成或曲面重网格化。本实验中输出 `Texcoords` 和二维 `Flat` 网格，为后续基于 UV 的采样和编辑提供了数据基础。

8.3 非同胚于盘的情况

本实验主要使用带边界且可展开为盘的模型。若输入曲面不是同胚于盘，例如闭合曲面或带多个洞的曲面，单一参数域会遇到拓扑障碍，需要额外切割生成边界，或使用多 `chart` 参数化。实现中若边界不足，会退化为选择两个远点固定自由度并使用初始投影，但这不能从拓扑上保证无重叠展开。完整处理应包括自动 `cut graph`、`seam` 设计和多 `chart` 拼接。

9 AI 辅助说明

本实验使用 Codex 作为编程和报告辅助工具。AI 主要参与以下工作：

- 根据作业 README 和论文步骤梳理 ARAP local/global 框架；
- 辅助实现 `hw6_arap.cpp` 中的稀疏线性系统、局部旋转拟合、ASAP、Hybrid、翻转检测与指标输出；
- 协助定位构建与运行问题，例如第三方依赖下载失败、Eigen 链接问题、USD 读写节点导致的重复刷新；
- 根据助教上次评分反馈，补充报告结构、公式说明、实验结果解释、AI 使用分析和附录记录。

AI 输出并非直接无审查采用。实现过程中，我通过 `Ruzino` 程序运行、节点连线测试、不同迭代次数和不同参数的截图验证算法行为；当程序出现卡死时，结合控制台输出和代码路径判断问题来自缺失 UV 时的初始化求解，并将其改为更适合交互式场景的主平面投影初始化。

10 总结

本实验完成了 ARAP 参数化的基本 local/global 算法，并扩展实现了 ASAP、Hybrid、并行 local phase、翻转后处理与定量指标输出。实验结果显示，ARAP 能通过迭代逐步改善初始参数化，ASAP、Hybrid、ARAP 在局部变换约束上的差异会带来不同的展开形态。通过稀疏矩阵预分解、解析二维旋转拟合和缓存机制，程序在交互式节点环境下具有更好的稳定性和效率。

11 附录：AI 辅助过程记录

本附录记录本次实验中与 Codex 的主要交互内容。为避免无关环境输出占据过多篇幅，以下按时间顺序整理为摘要式聊天记录，保留关键问题、修改方向和验证结论。

11.1 作业理解与实现范围

用户：阅读 README.md 与原论文 arap.pdf，根据作业要求完成 HW6；只需要改代码，不需要启动程序，不需要写实验报告；完成基础任务、可选任务和拓展任务。

Codex：阅读作业文档和论文后，确认实现目标包括初始参数化、ARAP local phase、global phase、迭代求解、ASAP、Hybrid、并行 local phase、翻转处理和误差指标。随后在 hw6_arap.cpp 中实现相关节点与算法。

11.2 构建依赖问题

用户：CMake 配置时无法从 GitHub 克隆 libigl/triangle.git，导致 triangle-populate 失败。

Codex：分析为网络导致的第三方依赖下载失败，添加本地 triangle_fallback，并修改 CMake recipe，使构建优先使用本地 fallback，避免依赖在线下载。

用户：链接 hw6_arap.dll 时出现 Eigen inverse 和 determinant 未解析外部符号。

Codex：定位为 Eigen 小矩阵分解相关头文件未完整包含，补充 #include <Eigen/LU>，解决链接问题。

用户：生成 Python stub 时 RHI_py 和 geometry_py 导入失败，提示 DLL load failed。

Codex：分析为 DLL 搜索路径缺失，在 nanobind stubgen 启动路径中补充 Ruzino 的 Debug binaries、SDK Python、Slang、OpenUSD、Embree 等 DLL 目录。

11.3 程序启动与节点使用

用户：编译后不知道 exe 在哪里，运行 .\Binaries\Debug\Ruzino.exe 没反应。

Codex: 说明可执行文件位于 Framework3D/Ruzino/Binaries/Debug/Ruzino.exe, 直接从普通终端运行可能缺少 DLL 搜索路径; 需要进入正确目录并设置 PATH, 或从 VS 调试环境启动。

用户: 询问 USD 的 Prim Path 应如何填写。

Codex: 根据 Stage Viewer 与 USD 文件结构说明, 当前 Stage 里的 /mesh_0 不一定是文件内部 prim path; 例如 Cow 文件通常应使用 /Cow_dABF/Cow_dABF, Beetle 文件使用对应模型名路径。

用户: 询问 HW6 需要连接哪些节点。

Codex: 给出基础连接方式: `read_usd.Geometry -> hw6_arap.Input, read_usd.Geometry -> hw6_arap.Original Mesh, hw6_arap.Flat -> write_usd.Geometry`, 并建议 `write_usd.Sub Path` 使用非空新名字以避免覆盖原始 prim。

11.4 卡死问题定位与优化

用户: 照节点连接后程序卡死, 怀疑算法计算量过大。

Codex: 重新检查论文步骤与代码路径, 发现输入的 `*_ABF.usda / *_dABF.usda` 在当前读取节点中没有得到可用逐顶点 UV, 导致 HW6 每次执行都进入 harmonic 初始参数化分支; 该分支需要全局稀疏求解, 在 UI 实时节点图中容易造成长时间卡顿。

Codex: 优化如下:

- 缺少 UV 时改用三维主平面投影作为初始参数化, 复杂度为 $O(n)$;
- ARAP local phase 中最近旋转由通用 SVD 改为二维解析极分解;
- global phase 使用稀疏矩阵和 `SimplicialLDLT` 预分解;
- 固定点选择由全点对搜索改为两次 `farthest scan`;
- 为 `write_usd` 增加缓存, 减少重复写 USD 导致的 Stage dirty 循环;
- 为 HW6 节点增加求解结果缓存, 避免同一输入和参数重复计算。

用户: 再次测试后程序可以正常运行。

11.5 报告截图与定量指标

用户: 询问为了测试 HW6 效果, 需要在程序中进行哪些测试、截哪些图片。

Codex: 建议截图包括节点图、初始参数化、ARAP 迭代 1/5/20 次、ASAP/Hybrid/ARAP 对比、Hybrid 的 λ 消融、棋盘纹理、翻转后处理、定量指标输出、并行 local phase 对比等。

用户: 指出 hw6_arap 节点没有 Mode 参数。

Codex: 纠正说明: Mode 只存在于统一入口 hw6_parameterization; 专用节点 hw6_arap、hw6_asap、hw6_hybrid 分别固定对应模型, 因此截图时只需调节各自节点的 Iterations、Lambda、Parallel Local Phase 与 Postprocess Iterations。

用户: 询问在哪里查看 Energy 等指标。

Codex: 说明这些是节点输出端口, 不是输入参数; 为方便报告截图, 在 hw6_arap.cpp 求解结束处加入 `spdlog::info`, 自动打印 energy、angle_distortion、area_distortion、flips_before、flips_after 和固定顶点编号。

11.6 报告写作

用户: 结合上一次报告模板写 HW6 报告, 并针对助教上次扣分点补强。

Codex: 根据上次评分反馈, 在报告中重点补充:

- 清晰的章节结构: 算法原理、实现细节、实验结果、结果分析、思考题与 AI 辅助说明;
- 公式化说明: ARAP 能量、局部旋转拟合、全局稀疏线性系统、ASAP 和 Hybrid 模型;
- 实验结果展示: 迭代消融、方法对比、参数消融、棋盘纹理、翻转后处理、定量指标;
- 程序实现说明: 稀疏矩阵、预分解、缓存、并行 local phase、生命周期管理;
- AI 使用分析: 说明 AI 的作用、人工验证过程与对 AI 结果的筛选。